

Unit - 4 : Collections

1. Introduction of Collection

- Collection एक framework है जो multiple objects (data) को single unit में store और manage करने के लिए use होता है।
- Collection का उपयोग data को organize, access, modify और process करने के लिए किया जाता है।
- यह Java में java.util package का part है और Java Collections Framework (JCF) के अंदर आता है।
- Collection data structures provide करता है जो dynamic और flexible होते हैं, यानी runtime पर size change हो सकती है।
- यह arrays से ज्यादा powerful होता है क्योंकि array का size fix होता है, जबकि collection में elements को आसानी से add, remove या modify किया जा सकता है।
- Collection में हम primitive data types (int, char, etc.) directly store नहीं कर सकते, यहाँ केवल objects store होते हैं। इसलिए primitive types के लिए wrapper classes (Integer, Character, आदि) का use किया जाता है।

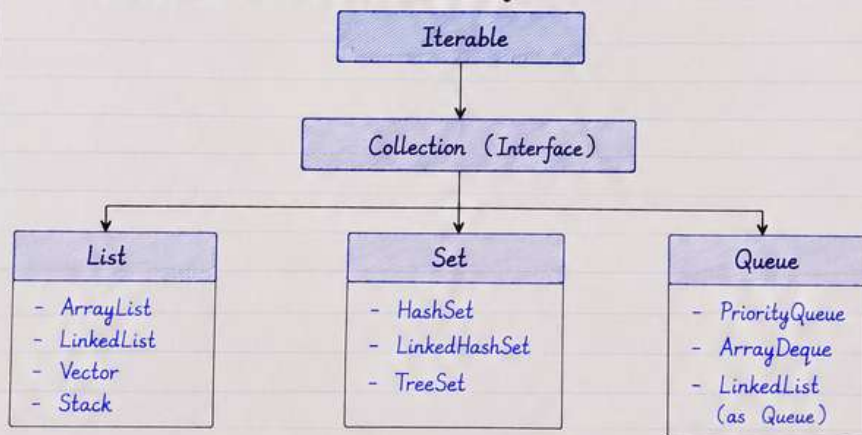
2. Why do we need Collection ?

- Multiple objects को एक साथ store करने के लिए।
- Data को efficient तरीके से manage करने के लिए।
- Dynamic size की आवश्यकता होती है (runtime पर size बदल सकता है)।
- Useful built-in methods provide करता है जैसे add(), remove(), search(), sort() आदि।
- Code को reusability और maintainability देता है।
- Real-world applications में data को handle करने के लिए collection बहुत useful है।

Key Point

- Collection = Group of Objects
- Part of java.util package
- Provides dynamic, flexible and efficient way to store and manage data.

3. Collection Framework Hierarchy



Note :

- Iterable is the root interface.
- Collection extends Iterable.
- List, Set और Queue Collection के main sub-interfaces हैं।
- इन interfaces के कई implementations (classes) होते हैं, जिन्हें हम requirement के अनुसार use करते हैं।

4. Important Interfaces with Short Description

Interface	Description
Collection	यह root interface है जो सभी collections के लिए common methods define करता है।
List	Ordered collection, duplicate elements allow करता है।
Set	Unique elements store करता है (duplicate allow नहीं)।
Queue	FIFO (First In First Out) order में elements store करता है।
Map (not child of Collection)	Key-Value pairs store करता है (यह Collection interface को extend नहीं करता)।

5. Common Methods (Collection Interface)

Method	Description
add(E e)	Collection में element add करता है।
remove(Object o)	Specified element को remove करता है।
size()	Total number of elements return करता है।
isEmpty()	Check करता है कि collection empty है या नहीं।
contains(Object o)	Check करता है कि element present है या नहीं।
clear()	सभी elements को remove करता है।
iterator()	Elements को iterate करने के लिए iterator return करता है।

6. Advantages of Collection

- Dynamic size
- Built-in useful methods
- Easy to use and implement
- Increases code reusability
- Better performance and efficient data handling
- Widely used in real-world applications

7. Summary

- Collection Java में objects को store और manage करने का एक powerful framework है, जो developer को data को efficiently handle करने में मदद करता है।

Topic : What is List ? 7 Major Feature of List

1. What is List ?

- List एक ordered collection (क्रमबद्ध संग्रह) होता है, जिसमें multiple elements को एक single unit में store किया जाता है।
- List आमतौर पर dynamic होती है, यानी इसका size runtime पर बदल सकता है।
- List की सबसे बड़ी खासियत यह है कि यह elements को insertion order (जिस क्रम में add किया गया है) उसी क्रम में maintain करती है।
- Java में List एक interface है जो java.util package के अंतर्गत आता है और यह Collection Framework का part है।
- List duplicate elements को allow करती है, और same element multiple times store किया जा सकता है।
- List index के through access provide करती है, जिससे हम किसी भी element को उसके position (index) द्वारा आसानी से प्राप्त कर सकते हैं।

List Representation (Example)

Index →	0	1	2	3	4
Elements →	10	20	30	40	50

List stores elements in an ordered way and each element can be accessed using its index.

Key Point

- List = Ordered collection
- Part of Collection Framework
- Maintains insertion order
- Allows duplicate elements
- Index based access
- Dynamic in nature

2. 7 Major Features of List

(1) Ordered Collection

- List elements को एक निश्चित क्रम (insertion order) में store करती है।
- हम जिस order में elements add करते हैं, उसी order में यह maintain रहती है।

Example : [10, 20, 30, 40]
(Same order will be maintained)

(2) Allows Duplicates

- List में duplicate elements allow होते हैं।
- एक ही value को multiple times add किया जा सकता है।

Example : [10, 20, 10, 30, 20]

(3) Index Based Access

- List में हर element का एक index (position) होता है, जो 0 से start होता है।
- हम index के द्वारा किसी भी element को आसानी से access कर सकते हैं।

Example : List.get(2) → तीसरा element प्राप्त होगा।

(4) Dynamic Size

- List का size fixed नहीं होता।
- हम elements को add या remove करके इसका size बढ़ा या घटा सकते हैं।
- यह runtime पर automatically manage होती है।

(5) Supports Multiple Data Types

- List में विभिन्न प्रकार के data types के elements store किए जा सकते हैं (Integer, String, Object, etc.)
- यह generic के साथ type safe भी बन सकती है।

Example : List<String> names = new ArrayList<>();

(6) Provides Useful Methods

- List interface कई built-in methods provide करती है, जैसे add(), remove(), get(), set(), size(), contains(), indexOf(), आदि।
- ये methods data को manage करने में बहुत मदद करते हैं।

(7) Part of Collection Framework

- List, Java Collection Framework का एक important interface है (java.util package में)।
- इसके कुछ common implementations हैं:

ArrayList, LinkedList, Vector, Stack
(ये सभी List interface को implement करते हैं।)

Advantages of List

- Data को ordered manner में store करती है।
- Duplicate elements allow करती है।
- Index based access provide करती है।
- Dynamic और flexible होती है।
- Built-in methods के कारण operations आसान हो जाते हैं।

Conclusion

List एक powerful collection है जो ordered, dynamic और flexible होती है। यह duplicate elements को allow करती है और index के द्वारा access provide करती है। Java programming में List का उपयोग बहुत व्यापक रूप से किया जाता है।

Topic : Modifying Lists

- List ek mutable data structure hai, yani iske elements ko modify (change) kiya jã sakta hai।
- Modifying ka matlab existing elements ko update karna, naye elements add karna, ya multiple elements ko ek saath change karna hota hai।

1. Multiple elements change karna

- List me ek se zyada elements ko ek saath change karne ke लिए indexing aur slicing ka use kar सकते हैं।
- Isme hum specific range ke elements ko nayi values se replace kar सकते हैं।

Example 1 : Indexing se multiple change

```
# original list
marks = [10, 20, 30, 40, 50]
# multiple elements change karna
marks[1] = 25
marks[3] = 45
print(marks)
# Output: [10, 25, 30, 45, 50]
```

Example 2 : Slicing se multiple change

```
nums = [1, 2, 3, 4, 5, 6]
# index 1 se 3 tak ke elements change
nums[1:4] = [20, 30, 40]
print(nums)
# Output: [1, 20, 30, 40, 5, 6]
```

Key Points

- Index se single element change hota hai।
- Slicing se multiple elements change kiye जा सकते हैं।
- Naya list ka size same ho zaruri nahi hai।

2. append()

- append() method list ke end me ek single element ko add करता है।
- Yeh existing list me naya element jodta है और original list ko modify करता है।

Syntax

```
list.append(element)
```

Example

```
fruits = ['apple', 'banana', 'mango']
fruits.append('orange')
print(fruits)
# Output: ['apple', 'banana', 'mango', 'orange']
```

Key Points

- append() sirf ek element add करता है।
- Yeh element hamesha list ke end me add hota है।
- Original list modify hoti है।

3. insert()

- insert() method list me kisi specific index (position) par element ko add करता है।
- Isse hum element ko list ke beech me bhi insert kar सकते हैं।

Syntax

```
list.insert(index, element)
```

Example

```
numbers = [10, 20, 30, 40]
numbers.insert(2, 25) # index 2 par 25 insert
print(numbers)
# Output: [10, 20, 25, 30, 40]
```

Key Points

- insert() me index dena jaruri hota है।
- Element us index par add hota है, aur baaki elements apni jagah se shift ho jate हैं।

4. extend()

- extend() method ek list ke multiple elements ko dusri iterable (list, tuple, set etc.) se add करता है।
- Yeh append() se alag hota है, kyunki append() ek single element add करता है, jabki extend() puri iterable ke sabhi elements ko add करता है।

Syntax

```
list.extend(iterable)
```

Example

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
list1.extend(list2)
print(list1)
# Output: [1, 2, 3, 4, 5, 6]
```

Key Points

- extend() multiple elements add करता है।
- Argument ek iterable (list, tuple, set, etc.) hota है।
- Original list modify hoti है।
- Agar hum list ko append() se add karen, to poori list ek element ke रूप में जुड़ जाएगी।

append() vs extend()

```
l = [1, 2, 3]
l.append([4, 5]) # [1, 2, 3, [4, 5]]
l = [1, 2, 3]
l.extend([4, 5]) # [1, 2, 3, 4, 5]
```

Note :

- List mutable hoti है, isliye in methods se latest changes original list par hi apply होते हैं।
- In methods ka use karke hum data ko easily manage aur modify kar सकते हैं।

Topic : Removing elements (remove(), pop(), del, clear())

- List se unwanted elements ko remove karne ke liye Python me kai methods aur statements available hote hain.
- Hum element ko value se, index se, ya puri list ko ek saath bhi remove kar sakte hain.
- Yeh operations list ko modify (change) karte hain, isliye yeh original list par hi apply hote hain.

1. remove()

- remove() method list me diye gaye value (element) ke first occurrence ko remove karta hai.
- Agar value list me multiple times hai, to sirf pehla element remove hota hai.
- Agar value list me nahi milti, to ValueError aata hai.

Syntax :

```
list.remove(value)
```

Example :

```
fruits = ['apple', 'banana', 'apple', 'mango']
fruits.remove('apple')
print(fruits)
# Output: ['banana', 'apple', 'mango']
```

2. pop()

- pop() method default roop se list ke last element ko remove karta hai aur us element ko return bhi karta hai.
- Agar index diya jaye, to us index par maujood element ko remove karke return karta hai.

Syntax :

```
list.pop()      # last element remove
list.pop(index) # specific index element remove
```

Example :

```
nums = [10, 20, 30, 40]
x = nums.pop()      # last element remove
print(x)            # Output: 40
print(nums)         # Output: [10, 20, 30]
y = nums.pop(1)     # index 1 par element remove
print(y)            # Output: 20
print(nums)         # Output: [10, 30]
```

3. del (delete statement)

- del ek Python statement hai, jo list ke kisi specific index ke element ko remove karne ke liye use hota hai.
- Isse single element, multiple elements (slicing) ya poori list bhi delete ki ja sakti hai.

Syntax :

```
del list[index]
del list[start : end]
del list      # puri list delete
```

Example :

```
nums = [10, 20, 30, 40, 50]
del nums[2]      # index 2 (30) remove
print(nums)     # [10, 20, 40, 50]
del nums[1:3]    # index 1 se 2 tak remove
print(nums)     # [10, 50]
# del nums      # puri list delete (phir use nahi kar sakte)
```

4. clear()

- clear() method list ke sabhi elements ko remove kar deta hai, lekin list khud delete nahi hoti.
- Operation ke baad list empty ho jati hai (size = 0).

Syntax :

```
list.clear()
```

Example :

```
colors = ['red', 'green', 'blue']
colors.clear()
print(colors)      # Output: []
print(len(colors)) # Output: 0
```

Note :

- remove() → value se element remove karta hai (first occurrence).
- pop() → index se element remove karta hai aur return karta hai.
- del → index, slice ya puri list delete kar sakte hain.
- clear() → sirf list ko empty karta hai (list rehti hai).

5. Difference between remove(), pop(), del and clear()

Method / Statement	Kis basis par remove hota hai?	Return value	Multiple elements remove kar sakte hain?	List rehti hai?	Error (agar element nahi mila)
remove()	Value (first occurrence)	No	No (sirf pehla)	Yes	ValueError
pop()	Index (default last)	Yes (removed element)	No (ek baar me ek)	Yes	IndexError (invalid index)
del	Index / Slice / Puri list	No	Yes (slicing se)	Yes (unless full list delete ki)	IndexError (invalid index)
clear()	Sabhi elements	No	Yes (all at once)	Yes (empty list)	No error

Topic : List Operators

List operators ka use lists ke saath different operations perform karne ke liye hota hai. In operators ke dwara hum lists ko combine, repeat, compare aur check kar sakte hain.

1. + (Concatenation)

- '+' operators do lists ko jod (concatenate) karne ke liye use hota hai.
- Isse ek nayi list banti hai jisme dono lists ke elements same order me aate hain.

Example :

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
list3 = list1 + list2 # concatenate
print(list3)
# Output : [1, 2, 3, 4, 5, 6]
```

2. * (Repetition)

- '*' operator list ke elements ko specified number of times repeat karne ke liye use hota hai.
- Isse ek nayi list banti hai jisme original list ke saare elements multiple times hote hain.

Example :

```
list1 = [10, 20, 30]
list2 = list1 * 3 # repeat 3 times
print(list2)
# Output : [10, 20, 30, 10, 20, 30, 10, 20, 30]
```

3. in (Membership Operator)

- 'in' operator check karta hai ki koi element list me present hai ya nahi.
- Agar element list me milta hai to True return hota hai warna False.

Example :

```
fruits = ['apple', 'banana', 'mango']
print('banana' in fruits) # True
print('orange' in fruits) # False
```

4. not in (Membership Operator)

- 'not in' operator check karta hai ki koi element list me present nahi hai.
- Agar element list me nahi hota to True return hota hai warna False.

Example :

```
fruits = ['apple', 'banana', 'mango']
print('orange' not in fruits) # True
print('banana' not in fruits) # False
```

5. ==, != (Equality Operators)

- '==' operator do lists ki equality check karta hai.
- Yeh True return karta hai agar dono lists ke elements same order me aur same values ke saath hote hain.
- '!=' operator True return karta hai agar dono lists same nahi hote.

Example :

```
list1 = [1, 2, 3]
list2 = [1, 2, 3]
list3 = [3, 2, 1]
print(list1 == list2) # True
print(list1 != list3) # True
```

6. <, > (Comparison Operators)

- '<' aur '>' operators lists ko lexicographical (dictionary) order me compare karte hain.
- Comparison left se right hota hai, jahan pehla difference milta hai uske basis par result decide hota hai.
- Yeh operators tab use hote hain jab lists ke elements comparable (jaise numbers ya strings) hote hain.

Example :

```
list1 = [1, 2, 3]
list2 = [1, 2, 4]
print(list1 < list2) # True (3 < 4)
print(list2 > list1) # True
list3 = [1, 2, 3]
print(list1 < list3) # False (both equal)
```

Summary Table

Operator	Name	Purpose	Example	Result
+	Concatenation	Do lists ko jodna	[1,2] + [3,4]	[1,2,3,4]
*	Repetition	List ko repeat karna	[1,2] * 3	[1,2,1,2,1,2]
in	Membership	Element present hai?	3 in [1,2,3]	True
not in	Membership	Element present nahi hai?	5 not in [1,2,3]	True
==	Equality	Dono lists same?	[1,2] == [1,2]	True
!=	Inequality	Dono lists different?	[1,2] != [2,1]	True
<	Less than	Lexicographical compare	[1,2] < [1,3]	True
>	Greater than	Lexicographical compare	[2,1] > [1,9]	True

Key Points :

- '+' aur '*' se hamesha nayi list banti hai, original list change nahi hoti.
- 'in' aur 'not in' membership check ke liye use hote hain.
- '==' aur '!=' order ke saath compare karte hain.
- '<' aur '>' lexicographical order me comparison karte hain.
- In operators ka use real-world applications me data ko search, combine aur compare karne ke liye hota hai.

Topic : Important List Functions

- Python me list ke sath kaam karne ke liye kai built-in functions available hote hain.
- In functions ka use list se related common operations ko easily perform karne ke liye hota hai.
- Yeh functions time bachate hain aur code ko simple aur readable banate hain.

1. len()

- len() function list me total number of elements (length) batata hai.
- Yeh hume list ka size (kitne elements hain) janane ke liye use hota hai.

Example :

```
nums = [10, 20, 30, 40, 50]
print(len(nums))    # Output : 5
fruits = ['apple', 'banana', 'mango']
print(len(fruits))  # Output : 3
```

2. max()

- max() function list me se sabse badi value (maximum element) return karta hai.
- Yeh mainly numeric data ke liye use hota hai, lekin strings me bhi lexicographical order ke basis par kaam karta hai.

Example :

```
nums = [10, 50, 30, 20]
print(max(nums))    # Output : 50
names = ['ram', 'shyam', 'mohan']
print(max(names))   # Output : 'shyam'
```

3. min()

- min() function list me se sabse chhoti value (minimum element) return karta hai.
- Yeh bhi numeric data ke liye use hota hai aur strings me lexicographical order follow karta hai.

Example :

```
nums = [10, 5, 30, 20]
print(min(nums))    # Output : 5
names = ['ram', 'shyam', 'mohan']
print(min(names))   # Output : 'mohan'
```

4. sum()

- sum() function list ke sabhi numeric elements ka total (sum) return karta hai.
- Yeh sirf numbers (int, float) ke liye work karta hai.
- Agar list me non-numeric value hogi to TypeError aayega.

Example :

```
nums = [10, 20, 30, 40]
print(sum(nums))    # Output : 100
marks = [78, 82, 90, 85]
print(sum(marks))   # Output : 335
```

5. sorted()

- sorted() function list ke elements ko sorted order (ascending by default) me return karta hai.
- Yeh original list ko modify nahi karta, balki ek nayi sorted list return karta hai.

Example :

```
nums = [40, 10, 30, 20]
print(sorted(nums)) # Output : [10, 20, 30, 40]
names = ['mango', 'apple', 'banana']
print(sorted(names)) # Output : ['apple', 'banana', 'mango']
print(nums)          # Original list unchanged
                     # Output : [40, 10, 30, 20]
```

6. reversed()

- reversed() function list ke elements ko reverse order me return karta hai.
- Yeh ek iterator return karta hai, isliye result ko dekhne ke liye hume use list() me convert karna padta hai.

Example :

```
nums = [10, 20, 30, 40]
rev = reversed(nums)
print(list(rev))    # Output : [40, 30, 20, 10]
names = ['a', 'b', 'c']
print(list(reversed(names))) # Output : ['c', 'b', 'a']
```

7. list()

- list() function ka use kisi other iterable (jaise string, tuple, set, range, etc.) ko list me convert karne ke liye hota hai.
- Yeh nayi list banata hai.

Example :

```
s = 'hello'
print(list(s))      # Output : ['h', 'e', 'l', 'l', 'o']
t = (1, 2, 3)
print(list(t))      # Output : [1, 2, 3]
r = range(1, 6)
print(list(r))      # Output : [1, 2, 3, 4, 5]
```

Key Points :

- len() → list ki length batata hai.
- max() → sabse badi value return karta hai.
- min() → sabse chhoti value return karta hai.
- sum() → sabhi elements ka total deta hai.
- sorted() → sorted nayi list return karta hai (original list change nahi hoti).
- reversed() → elements ko reverse order me deta hai (iterator return karta hai).
- list() → kisi iterable ko list me convert karta hai.
- Yeh functions list par kaam karna simple, efficient aur easy bana dete hain.

Topic : Tuples

- Tuple Python में एक built-in data type है, जो multiple elements को
- एक single unit में store करने के लिए use होता है।
- Tuple ordered collection होती है और immutable (जिसे change नहीं किया जा सकता) होती है।
- Tuple में different data types के elements store किए जा सकते हैं।
- Tuple को round brackets () में लिखा जाता है और elements comma (,) से separate होती है।

1. Creating a Tuple

- Tuple को directly values देकर बनाया जाता है।
- Single element tuple के लिए value के बाद comma (,) लगाना जरूरी होता है, नहीं तो वह tuple नहीं माना जाएगा।

Example :

```
t1 = (10, 20, 30)           # tuple of integers
t2 = ("apple", "banana", "mango") # tuple of strings
t3 = (10, "Hello", 3.5, True) # mixed data types
t4 = (50,)                  # single element tuple
t5 = ()                     # empty tuple

print(t1)                   # (10, 20, 30)
print(type(t1))             # <class 'tuple'>
```

4. Tuple Operations

- Tuple पर कई operations perform किए जा सकते हैं जैसे concatenation (+), repetition (*), membership (in), length (len()), आदि।

Example :

```
t1 = (1, 2, 3)
t2 = (4, 5)
print(t1 + t2) # (1, 2, 3, 4, 5) (concatenation)
print(t1 * 2)  # (1, 2, 3, 1, 2, 3) (repetition)
print(2 in t1) # True (membership)
print(len(t1)) # 3 (length)
```

6. Tuple Methods

- Tuple में list की तरह change करने वाले methods नहीं होते, क्योंकि यह immutable है।
- लेकिन इसमें कुछ useful methods available होते हैं।

Method	Description	Example
count(x)	कितनी बार value आती है, यह return करता है।	t.count(10)
index(x)	पहली occurrence का index return करता है।	t.index(20)

9. When to Use Tuple ?

- जब data change नहीं होना चाहिए तब Tuple use करें।
- Functions से multiple values return करने के लिए।
- Fixed data collection के लिए (जैसे coordinates, days of week, etc.)
- Data की integrity बनाए रखने के लिए।

2. Accessing Tuple Elements

- Tuple के elements को index के द्वारा access किया जाता है।
- Indexing 0 से start होती है और negative indexing भी use की जा सकती है।

Example :

```
t = (10, 20, 30, 40, 50)
print(t[0])    # 10
print(t[2])    # 30
print(t[-1])   # 50 (last element)
print(t[-2])   # 40
```

3. Tuple Slicing

- Tuple में slicing का use करके elements की range प्राप्त की जा सकती है।

Example :

```
t = (10, 20, 30, 40, 50, 60)
print(t[1:4])  # (20, 30, 40)
print(t[:3])   # (10, 20, 30)
print(t[3:])   # (40, 50, 60)
print(t[::2])  # (10, 30, 50)
```

5. Immutability of Tuple

- Tuple immutable होती है, यानी एक बार बन जाने के बाद इसके elements को change, add या remove नहीं किया जा सकता।
- अगर हम किसी element को modify करने की कोशिश करते हैं, तो TypeError आता है।

Example :

```
t = (10, 20, 30)
t[0] = 100 # TypeError: 'tuple' object does not support
            # item assignment
```

Note :

- Tuple immutable होने के कारण यह data को secure रखने के लिए useful होती है।
- इसे functions में multiple values return करने के लिए भी use किया जाता है।

7. Advantages of Tuple

- Immutable होने के कारण data safe रहता है।
- List की तुलना में memory efficient होती है।
- Functions में multiple values return करने के लिए useful है।
- Heterogeneous (different data types) data store कर सकती है।

8. Disadvantages of Tuple

- Elements को change, add या remove नहीं कर सकते।
- Dynamic operations के लिए suitable नहीं है (list बेहतर होती है)।
- कम methods available हैं।

Topic : Sets

- Set Python में एक built-in data type है, जो unique (distinct) elements का unordered collection होता है।
- Set में duplicate values नहीं होती हैं।
- Set mutable होता है, इसलिए इसमें elements को add या remove किया जा सकता है।
- Set का use membership testing, duplicate हटाने और mathematical set operations (union, intersection, etc.) के लिए किया जाता है।

Key Points :

- Unordered collection
- Unique elements only
- Mutable
- Indexed नहीं होता।
- {} (curly braces) में लिखा जाता है।
- Mixed data types allowed

1. Creating a Set

- Set को curly braces {} में elements रखकर बनाया जाता है।
- या फिर set() function का use किया जाता है।

Example :

```
s1 = {10, 20, 30, 40}
s2 = {'apple', 'banana', 'mango'}
s3 = set([1, 2, 3, 3, 2, 1]) # duplicates remove
print(s3) # Output : {1, 2, 3}
```

2. Accessing Elements

- Set indexed नहीं होता, इसलिए हम index के द्वारा directly element access नहीं कर सकते।
- हम केवल membership operator (in, not in) का use करके check कर सकते हैं कि कोई element set में है या नहीं।

Example :

```
s = {10, 20, 30}
print(20 in s) # True
print(40 in s) # False
```

3. Adding Elements (add, update)

- add() method का use एक single element को add करने के लिए होता है।
- update() method का use multiple elements (iterable) को add करने के लिए होता है।

Example :

```
s = {10, 20, 30}
s.add(40) # {10, 20, 30, 40}
s.update([50, 60]) # {10, 20, 30, 40, 50, 60}
print(s)
```

4. Removing Elements (remove, discard, pop, clear)

- remove() - specified element remove करता है (अगर element नहीं है तो Error आता है)।
- discard() - specified element remove करता है (अगर element नहीं है तो Error नहीं आता)।
- pop() - set से कोई भी एक random element remove करता है।
- clear() - set के सभी elements को remove कर देता है (खाली set)।

Example :

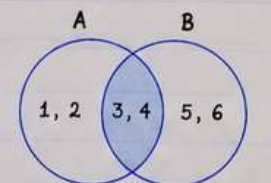
```
s = {10, 20, 30, 40}
s.remove(20) # {10, 30, 40}
s.discard(50) # No error
print(s.pop()) # removes and returns a random element
s.clear() # set() -> empty set
```

5. Set Operations

Operation	Symbol / Method	Description	Example
Union	or union()	सभी elements	A B
Intersection	& or intersection()	Common elements	A & B
Difference	- or difference()	A में है, B में नहीं	A - B
Symmetric Diff.	^ or symmetric_difference()	A या B में है, लेकिन दोनों में नहीं।	A ^ B

Example :

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B) # {1, 2, 3, 4, 5, 6}
print(A & B) # {3, 4}
print(A - B) # {1, 2}
print(A ^ B) # {1, 2, 5, 6}
```



A | B = {1, 2, 3, 4, 5, 6}
A & B = {3, 4}

6. Useful Set Functions

Function	Description	Example
len(s)	Total elements	len({1, 2, 3})
min(s)	Minimum element	min({3, 1, 2})
max(s)	Maximum element	max({3, 1, 2})
sum(s)	Sum of elements	sum({1, 2, 3})
sorted(s)	Sorted list (returns list)	sorted({3, 1, 2})
set(iterable)	Convert iterable to set	set([1, 2, 2, 3])

7. Properties of Set

- Unordered - elements ka order fix नहीं होता।
- Unique Elements - duplicate values नहीं होती।
- Mutable - elements को add या remove किया जा सकता है।
- Heterogeneous - different data types रख सकते हैं।
- Indexing नहीं - index के द्वारा element access नहीं कर सकते।
- Fast Membership Test - in और not in बहुत fast काम करते हैं।

8. When to Use Set ?

- जब हमें unique values store करनी हों।
- Duplicate values हटाने के लिए।
- Membership test (element है या नहीं) के लिए।
- Mathematical set operations के लिए (union, intersection, etc.)।
- Data को efficiently manage करने के लिए।
- List से duplicate remove करने के लिए -> set(List)।

Note :

- Set एक powerful data structure है, जो simple syntax के साथ data को unique और organized रखने में मदद करता है।

Topic : Dictionaries

- Dictionary Python में एक built-in data type है, जो data को key-value pairs के form में store करता है।
- Dictionary unordered (Python 3.7+ में insertion order maintain होती है), mutable और indexed (key के द्वारा) होती है।
- Dictionary का use real-world data को represent करने के लिए किया जाता है, जैसे student details, employee records, etc.
- Dictionary में keys unique होती हैं, लेकिन values duplicate हो सकती हैं।

1. Creating a Dictionary

- Dictionary को curly braces {} में key : value pairs लिखकर बनाया जाता है।
- Key किसी भी immutable type (string, number, tuple, etc.) की हो सकती है।

Example :

```
d1 = {'name': 'Rahul', 'age': 20, 'city': 'Delhi'}
d2 = { 1: 'one', 2: 'two', 3: 'three' }
d3 = dict(name = 'Sneha', age = 22)
print(d1) # {'name': 'Rahul', 'age': 20, 'city': 'Delhi'}
```

2. Accessing Elements

- Dictionary के elements को key के द्वारा access किया जाता है।
- अगर key exist नहीं करती तो KeyError आता है।

Example :

```
d = {'name': 'Rahul', 'age': 20, 'city': 'Delhi'}
print(d['name']) # Rahul
print(d['age']) # 20
print(d.get('city')) # Delhi
print(d.get('course', 'Not Found')) # Not Found
```

3. Adding and Updating Elements

- नया key-value pair add करने के लिए assignment operator (=) का use करते हैं।
- अगर key पहले से exist करती है तो उसकी value update हो जाती है।

Example :

```
d = {'name': 'Rahul', 'age': 20}
d['city'] = 'Delhi' # new key add
d['age'] = 21 # existing key update
print(d) # {'name': 'Rahul', 'age': 21, 'city': 'Delhi'}
```

4. Removing Elements

- किसी key-value pair को remove करने के लिए del, pop() या popitem() methods का use किया जाता है।

Example :

```
d = {'name': 'Rahul', 'age': 20, 'city': 'Delhi'}
del d['age'] # remove by key
d.pop('city') # remove and return value
d.popitem() # remove last inserted item
print(d) # {'name': 'Rahul'}
```

5. Useful Dictionary Methods

Method	Description	Example
keys()	सभी keys return करता है।	d.keys()
values()	सभी values return करता है।	d.values()
items()	सभी key-value pairs return करता है।	d.items()
get()	key की value return करता है (safe)।	d.get('name')
update()	दूसरे dictionary से update करता है।	d.update(d2)
pop()	दी गई key को remove करता है।	d.pop('age')
popitem()	आखिरी inserted item remove करता है।	d.popitem()
clear()	सभी items remove करता है।	d.clear()
copy()	Dictionary की एक copy बनाता है।	d.copy()

6. Dictionary Operations

- Dictionary पर membership operators (in, not in), length (len()), और comparison (==, !=) का use किया जा सकता है।

Example :

```
d = {'a': 10, 'b': 20, 'c': 30}
print('a' in d) # True
print('x' not in d) # True
print(len(d)) # 3
d2 = {'a': 10, 'b': 20, 'c': 30}
print(d == d2) # True
```

7. Nested Dictionary

- Dictionary के अंदर एक और dictionary भी हो सकती है।
- इसका use complex data (जैसे multiple students की details) के लिए किया जाता है।

Example :

```
students = {
    's1': {'name': 'Rahul', 'age': 20},
    's2': {'name': 'Sneha', 'age': 22}
}
print(students['s1']['name']) # Rahul
```

8. Advantages of Dictionary

- Data को key-value pairs में store करने के लिए efficient है।
- Fast access (key के द्वारा) मिलता है।
- Real-world data को represent करने के लिए useful है।
- Flexible और mutable होता है।

Key Points :

- Dictionary unordered, mutable और indexed (by key) होती है।
- Keys unique होती हैं, values duplicate हो सकती हैं।
- Key के द्वारा data को access, add, update और remove किया जाता है।
- Dictionary real-world applications में बहुत useful है।

Topic : Sorting Dictionaries

- Dictionary Python में key-value pairs का unordered collection होता है, इसलिए यह default रूप से sorted नहीं होता।
- कई बार हमें dictionary के keys या values को sorted order में देखना होता है।
- Sorting के लिए हम built-in functions जैसे sorted() और कुछ methods का use करते हैं।
- हम keys के अनुसार, values के अनुसार (या दोनों) dictionary को sort कर सकते हैं।

1. Sorting by Keys (Ascending Order)

- Dictionary को keys के अनुसार ascending order (A → Z) में sort करने के लिए sorted() का use किया जाता है।
- यह एक list of tuples return करता है, इसलिए चाहें तो इसे dict() में convert कर सकते हैं।

Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
sorted_d = dict(sorted(d.items()))
print(sorted_d)
# Output : {'apple': 5, 'banana': 3, 'mango': 1}
```

2. Sorting by Keys (Descending Order)

- Keys को descending order (Z → A) में sort करने के लिए sorted() में reverse = True देते हैं।

Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
sorted_d = dict(sorted(d.items(), reverse=True))
print(sorted_d)
# Output : {'mango': 1, 'banana': 3, 'apple': 5}
```

3. Sorting by Values (Ascending Order)

- Values के अनुसार sort करने के लिए sorted() में key parameter का use करते हैं (item[1] → value को consider करता है)।
- इससे dictionary value के increasing order में sort हो जाती है।

Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
sorted_d = dict(sorted(d.items(), key=lambda item: item[1]))
print(sorted_d)
# Output : {'mango': 1, 'banana': 3, 'apple': 5}
```

4. Sorting by Values (Descending Order)

- Values को descending order में sort करने के लिए reverse = True का use करते हैं।

Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
sorted_d = dict(sorted(d.items(),
    key=lambda item: item[1], reverse=True))
print(sorted_d)
# Output : {'apple': 5, 'banana': 3, 'mango': 1}
```

5. Using sorted() Without dict()

- अगर केवल sorted order में items को display करना है, तो dict() में convert करना जरूरी नहीं है।
- sorted() एक list of tuples return करता है।

Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
print(sorted(d.items()))
# Output : [('apple', 5), ('banana', 3), ('mango', 1)]
```

6. Sorting with Custom Logic

- हम अपनी जरूरत के अनुसार custom sorting भी कर सकते हैं जैसे keys की length के अनुसार, या values के किसी condition के अनुसार।

Example : (Keys की length के अनुसार sort)

```
d = {'python': 50, 'c': 70, 'java': 60}
sorted_d = dict(sorted(d.items(), key=lambda item: len(item[0])))
print(sorted_d)
# Output : {'c': 70, 'java': 60, 'python': 50}
```

Key Points :

- Dictionary default रूप से unordered होता है।
- sorted() function dictionary के items को sort करने के लिए use किया जाता है।
- Keys या values - दोनों के basis पर sorting की जा सकती है।
- sorted() हमेशा list of tuples return करता है।
- अगर sorted result को फिर से dictionary में चाहिए तो dict() का use करें।
- reverse = True देने से descending order में sort होता है।
- Custom logic के लिए lambda function का use किया जा सकता है।

Summary Example :

```
d = {'banana': 3, 'apple': 5, 'mango': 1}
# by keys (ascending)
print(dict(sorted(d.items())))
# by keys (descending)
print(dict(sorted(d.items(), reverse=True)))
# by values (ascending)
print(dict(sorted(d.items(), key=lambda x: x[1])))
# by values (descending)
print(dict(sorted(d.items(), key=lambda x: x[1], reverse=True)))
```


Topic : Copying Collections

- Python में collections (List, Tuple, Set, Dictionary) को copy करने के लिए different methods available हैं।
- Copying का use तब किया जाता है जब हमें original collection को modify किए बिना उसका एक duplicate बनाना होता है।
- अगर हम सिर्फ assignment operator (=) का use करते हैं, तो नया variable same object को refer करता है।
- इसलिए किसी एक में change करने पर दोनों में change आ जाता है।
- इस problem से बचने के लिए copy(), list(), dict(), set(), slice [:] आदि methods का use करते हैं।

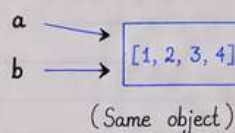
1. Assignment vs Copying (Important)

- = operator केवल reference copy करता है, नया object नहीं बनाता।
- दोनों variables एक ही memory location को refer करते हैं।

Example : (Assignment)

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)      # [1, 2, 3, 4]
print(b)      # [1, 2, 3, 4]
```

Memory Diagram :



Note : a और b दोनों एक ही object को refer कर रहे हैं, इसलिए किसी एक में change करने पर दोनों में change हो जाता है।

2. Copying a List

- List को copy करने के लिए copy(), list() या slicing [:] का use किया जा सकता है।
- यह methods एक नया independent list object बनाते हैं।

Example :

```
a = [10, 20, 30]
b = a.copy()      # using copy()
c = list(a)       # using list()
d = a[:]          # using slicing
b.append(40)
print(a)          # [10, 20, 30] (original unchanged)
print(b)          # [10, 20, 30, 40] (new list modified)
```

Note : a, b, c, d सभी अलग-अलग list objects हैं।

3. Copying a Tuple

- Tuple immutable होता है, इसलिए आमतौर पर copy करने की जरूरत नहीं होती।
- फिर भी हम tuple() constructor या slicing से इसका copy बना सकते हैं।

Example :

```
t1 = (1, 2, 3)
t2 = tuple(t1)
t3 = t1[:]
print(t2)      # (1, 2, 3)
print(t3)      # (1, 2, 3)
```

Note : Tuple immutable होता है, इसलिए copy करने पर भी data same रहता है, लेकिन नया object बना जाता है।

4. Copying a Set

- Set को copy करने के लिए copy() या set() constructor का use किया जाता है।
- इससे एक नया independent set object बनता है।

Example :

```
s1 = {1, 2, 3}
s2 = s1.copy()
s3 = set(s1)
s2.add(4)
print(s1)        # {1, 2, 3}
print(s2)        # {1, 2, 3, 4}
```

Note : s1 और s2 अलग-अलग set objects हैं।

5. Copying a Dictionary

- Dictionary को copy करने के लिए copy(), dict() या dict comprehension का use किया जा सकता है।
- इससे नया dictionary बनता है, जो original से independent होता है।

Example :

```
d1 = {'name': 'Rahul', 'age': 20}
d2 = d1.copy()
d3 = dict(d1)
d4 = {k: v for k, v in d1.items()}
d2['age'] = 22
print(d1)      # {'name': 'Rahul', 'age': 20}
print(d2)      # {'name': 'Rahul', 'age': 22}
```

Key Points :

- केवल reference copy करता है, नया object नहीं बनाता।
- copy(), list(), dict(), set(), [:] → new independent object बनाते हैं।
- Tuple immutable होता है, इसलिए copy की जरूरत कम होती है।
- Shallow copy केवल top-level copy करता है, Deep copy पूरा copy करता है।
- Copying का use तब करें जब original data को सुरक्षित रखना हो।

6. Shallow vs Deep Copy (Brief)

- Shallow Copy केवल top-level elements को copy करता है।
- अगर collection के अंदर भी कोई mutable object (जैसे list, dict) है, तो वह shared रहता है।
- Deep Copy पूरा object और उसके अंदर के भी सभी objects की copy बनाता है, ताकि कोई भी change original को affect न करे।

Example : (Nested List)

```
import copy
a = [[1, 2], [3, 4]]
b = a.copy()      # shallow copy
c = copy.deepcopy(a) # deep copy
b[0].append(10)
print(a)          # [[1, 2, 10], [3, 4]] (changed)
print(c)          # [[1, 2], [3, 4]] (unchanged)
```

Conclusion :

- Collections को copy करना important है, क्योंकि इससे हम data को modify करते समय original collection को सुरक्षित रख सकते हैं।
- Different methods अलग-अलग situations में useful होते हैं, इसलिए सही method का चयन करना चाहिए।